

JavaScript 异步编程

回调函数 (Callback)

回调函数是 JavaScript 中最基础的异步处理方式，将函数作为参数传给另一个函数，在异步操作完成后被调用。

回调地狱问题：多层嵌套的回调使代码难以阅读和维护。

Promise

Promise 是 ES6 引入的异步编程解决方案，解决回调地狱问题。

一个 Promise 有三种状态：pending（进行中）、fulfilled（已成功）、rejected（已失败）。状态一经改变，就不会再变。

async/await

async/await 是 ES7 引入的语法糖，基于 Promise 实现，让异步代码看起来像同步代码。async 函数返回一个 Promise。await 只能用在 async 函数内部。

事件循环 (Event Loop)

JavaScript 是单线程语言，通过事件循环实现非阻塞的异步。

宏任务 (macrotask) 如 setTimeout、I/O；微任务 (microtask) 如 Promise.then。

事件循环每轮先执行一个宏任务，再清空所有微任务。

Rust 内存安全

所有权 (Ownership)

Rust 的核心特性是所有权系统，确保内存安全而无需垃圾回收。

每个值有且仅有一个所有者，所有权可以转移 (move) 或借用 (borrow)。

借用与引用 (Borrowing)

借用用 `&` 符号表示，允许读取数据但不拥有数据。

可变借用 `&mut` 同一时间只能存在一个，避免数据竞争。

生命周期 (Lifetime)

生命周期是 Rust 编译器用来确保引用不会悬空的机制。

使用 `'a` 这样的标注来关联引用之间的生命周期关系。